

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Muhammad Rasyha Syauqi Islam - 5024241066

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1 : Pengenalan STM32 Blackpill (Blink)

1. Pastikan project Modul 1 sudah dibuat (Bagian D) dan isi platformio.ini.

Listing 1: Konfigurasi PlatformIO STM32 Blackpill dengan Framework Arduino

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Hubungkan ST-Link ke Blackpill dengan kabel, lalu colokkan ST-Link ke komputer lewat USB. Cocokkan nama pinnya, karena letak pin ST-Link bisa berbeda-beda:

Tabel 1: Koneksi Pin ST-Link dengan STM32 Blackpill

Pin ST-Link	Pin Blackpill	Fungsi
SWDIO	SWDIO	Jalur data/debug
SWCLK	SWCLK	Jalur clock/debug
GND	GND	Referensi ground bersama
3.3V	3V3	Referensi/tegangan target board

3. Tulis kode Blink di bawah pada src/main.cpp, lalu klik Build dan Upload.

Listing 2: Program Blink STM32 Blackpill

```
#include <Arduino.h>

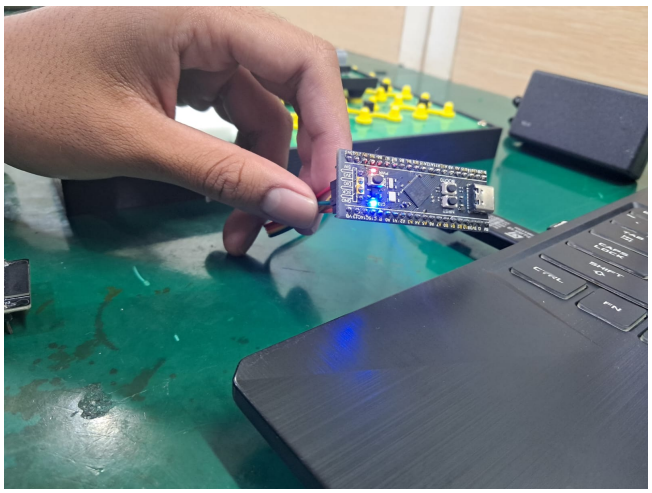
#define LED_PIN PC13

void setup()
{
    pinMode(LED_PIN, OUTPUT);
}

void loop()
{
    digitalWrite(LED_PIN, LOW);
    delay(500);

    digitalWrite(LED_PIN, HIGH);
    delay(500);
}
```

4. Lihat LED di board (PC13): nyala setengah detik, mati setengah detik, dan seterusnya.



Gambar 1: Hasil Percobaan 1

1.2 Percobaan 2 : Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Ganti isi platformio.ini dengan konfigurasi di atas (board ESP32 DevKit, framework ESP-IDF).

Listing 3: Konfigurasi PlatformIO untuk ESP32 dengan Framework ESP-IDF

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. Colokkan ESP32 ke komputer lewat USB, lalu pastikan komputer mengenali port-nya.

3. Hapus file src/main.cpp, buat file baru src/main.c, tulis kode Blink di bawah, lalu klik Build dan Upload.

4. Analisis apa perbedaan memprogram dengan ESP-IDF dan dengan Arduino.

Listing 4: Program Blink ESP32 dengan Framework ESP-IDF

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

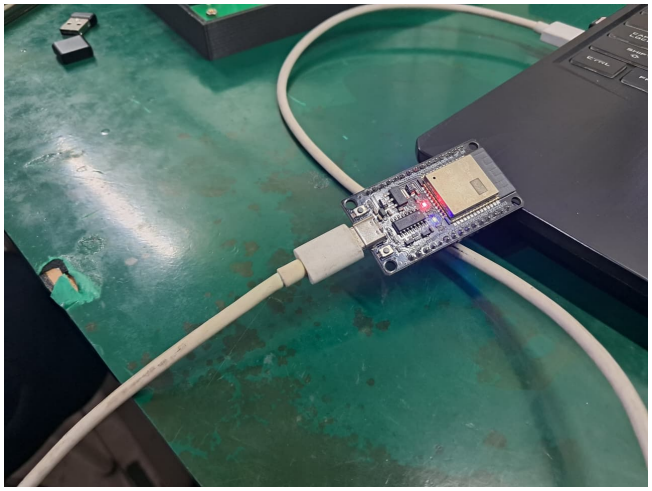
#define LED_PIN GPIO_NUM_2

void app_main(void)
{
    gpio_reset_pin(LED_PIN);
    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

    while (1)
    {
        gpio_set_level(LED_PIN, 1);
        vTaskDelay(pdMS_TO_TICKS(500));

        gpio_set_level(LED_PIN, 0);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
```

```
20 }
```



Gambar 2: Hasil Percobaan 2

1.3 Percobaan 3 : Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Ganti isi platformio.ini dengan konfigurasi di atas (board ESP32 DevKit, framework ESP-IDF).

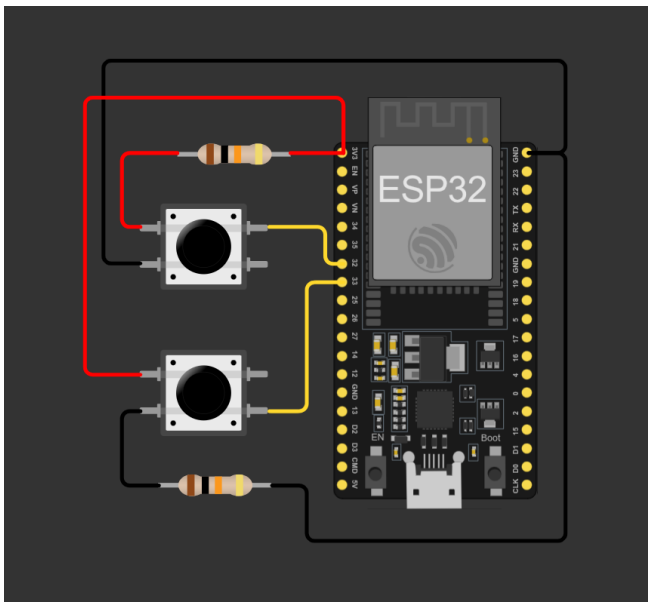
Listing 5: Konfigurasi PlatformIO ESP32 dengan Framework Arduino

```
1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = arduino
5 monitor_speed = 115200
```

2. Susunlah skema rangkaian, bagian A (Pull-up/Pull-down Eksternal):

Tabel 2: Konfigurasi Pushbutton pada ESP32

Komponen	Pin ESP32	Keterangan
Pushbutton 1 (pull-up eksternal)	GPIO 32	Satu kaki ke GPIO dan ke 3.3V melalui resistor 10kΩ (pull-up), kaki lain ke GND
Pushbutton 2 (pull-down eksternal)	GPIO 33	Satu kaki ke GPIO dan ke GND melalui resistor 10kΩ (pull-down), kaki lain ke 3.3V



Gambar 3: Skematik Rangkaian A

3. Tulis Kode Program A (Eksternal) di bawah pada src/main.cpp, klik Build dan Upload, lalu coba tekan kedua tombol dan catat hasilnya dari serial monitor.

Listing 6: Program Pengujian Pushbutton dengan Pull-up dan Pull-down Eksternal

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT);
}

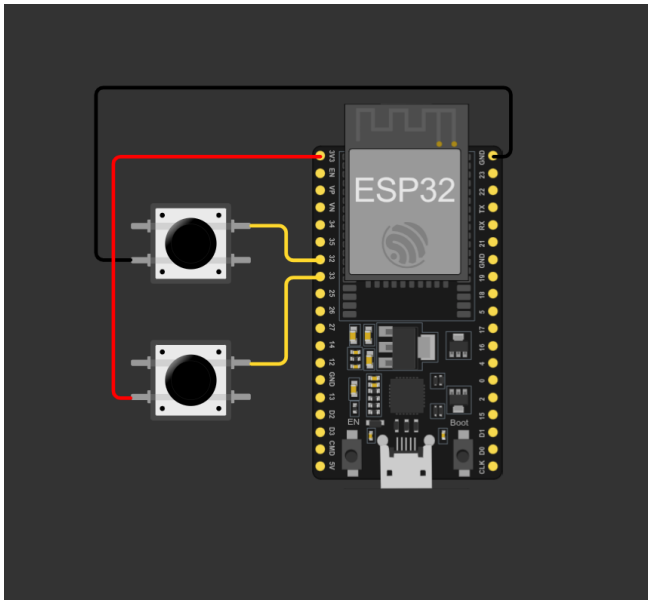
void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

4. Lepas kedua resistor sesuai Skema Bagian B (Gambar 7), tombol tetap terpasang. Lihat Gambar 8 dan Gambar 9 untuk memastikan pasangan kaki tombol yang benar.

Tabel 3: Konfigurasi Pushbutton dengan Pull-up dan Pull-down Internal

Komponen	Pin ESP32	Keterangan
Pushbutton 1 (pull-up internal)	GPIO 32	Satu kaki ke GPIO, kaki lain ke GND, gunakan `INPUT_PULLUP` internal, resistor eksternal dilepas
Pushbutton 2 (pull-down internal)	GPIO 33	Satu kaki ke GPIO, kaki lain ke 3.3V, gunakan `INPUT_PULLDOWN` internal, resistor eksternal dilepas



Gambar 4: Skematik Rangkaian B

5. Ganti isi src/main.cpp dengan Kode Program B (Internal) di bawah, klik Build dan Upload, lalu coba tombol yang sama dan catat hasilnya dari serial monitor.

Listing 7: Program Pengujian Pushbutton dengan Pull-up dan Pull-down Internal

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32
#define BUTTON_PULLDOWN_PIN 33

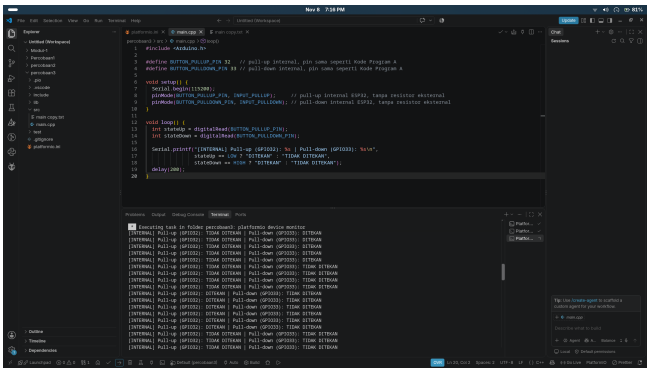
void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN);
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

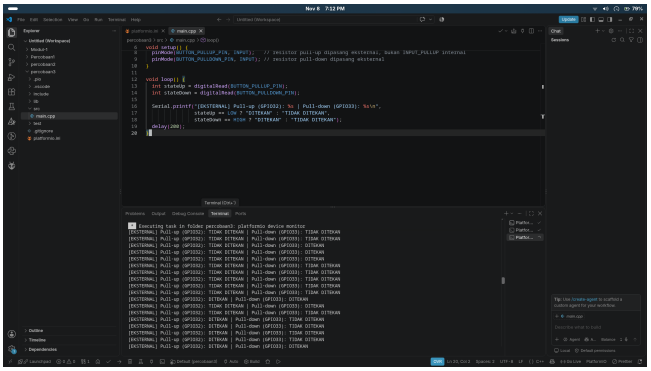
  Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

6. Bandingkan hasil A dan B. Seharusnya sama: pull-up bernilai HIGH saat tidak ditekan dan LOW saat ditekan, pull-down kebalikannya.

7. Jika hasilnya terbalik atau berbeda, periksa apakah kedua resistor sudah benar-benar dilepas.



Gambar 5: Hasil Percobaan



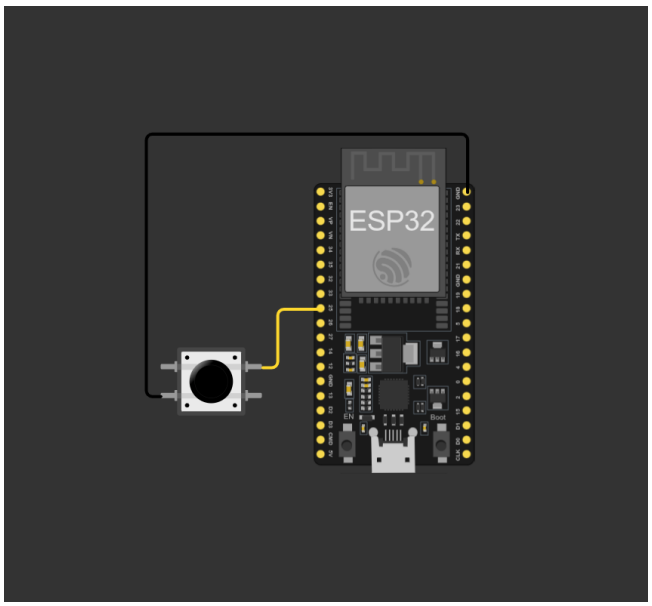
Gambar 6: Hasil Percobaan

1.4 Percobaan 4 : Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

1. Pasang tombol sesuai dengan skema rangkaian.

Tabel 4: Konfigurasi Pushbutton untuk Pengujian Debouncing

Komponen	Pin ESP32	Keterangan
Pushbutton (tactile)	GPIO 25	Satu kaki ke GPIO, kaki lain ke GND, gunakan `INPUT_PULLUP`



Gambar 7: Skematik Rangkaian

2. Tulis kode di bawah. Kode ini punya dua penghitung: counterNoDebounce (tanpa penyaring getaran tombol) dan counterWithDebounce (dengan penyaring).

Listing 8: Program Pengujian Debouncing pada Pushbutton ESP32

```
#include <Arduino.h>

#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50 // ms

// Tanpa debouncing
int rawLastState = HIGH;
unsigned long counterNoDebounce = 0;

// Dengan debouncing
int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  int reading = digitalRead(BUTTON_PIN);

  // ===== Tanpa Debouncing =====
  if (reading == LOW && rawLastState == HIGH) {
    counterNoDebounce++;
  }
  rawLastState = reading;

  // ===== Dengan Debouncing =====
```

```
  if (reading != lastFlickerableState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (reading != stableState) {
      stableState = reading;
      if (stableState == LOW) {
        counterWithDebounce++;
      }
    }
  }
  lastFlickerableState = reading;

  // ===== Tampilkan kedua nilai secara bersamaan =====
  static unsigned long lastPrintedNoDebounce = 0;
  static unsigned long lastPrintedWithDebounce = 0;

  if (counterNoDebounce != lastPrintedNoDebounce ||
      counterWithDebounce != lastPrintedWithDebounce) {
    Serial.printf("Tanpa Debounce: %lu\t| Dengan Debounce: %lu\n",
                  counterNoDebounce, counterWithDebounce);
    lastPrintedNoDebounce = counterNoDebounce;
    lastPrintedWithDebounce = counterWithDebounce;
  }
}
```

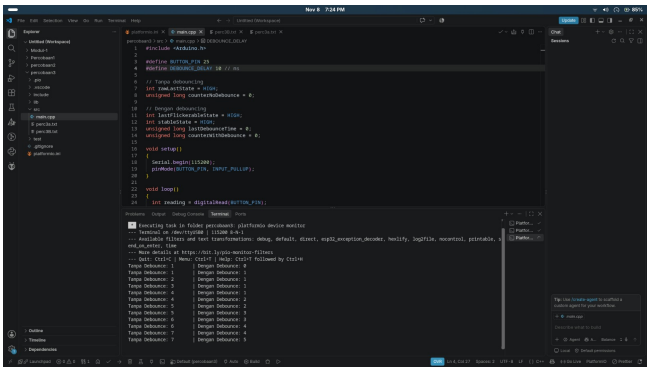
3. Tekan tombol satu kali, lalu bandingkan nilai kedua penghitung di Serial Monitor.

4. Ulangi dengan cara menekan yang berbeda (cepat dan lambat). counterNoDebounce bisa naik lebih dari 1 untuk satu tekanan, sedangkan counterWithDebounce seharusnya naik tepat 1. Jika keduanya terlihat sama, coba tekan lagi karena getaran tombol tidak selalu muncul.

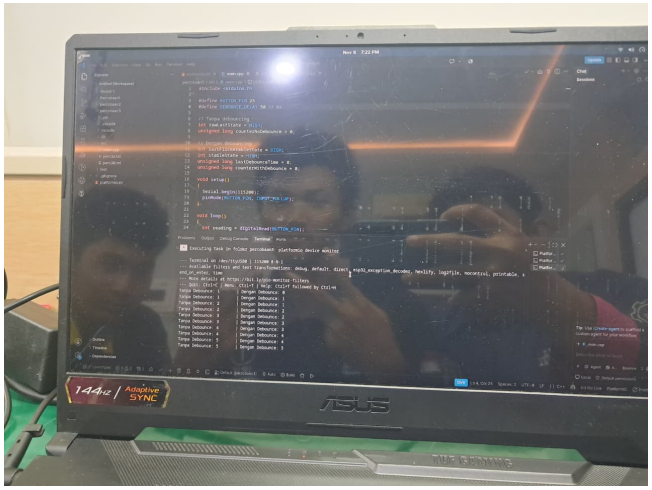
5. Ubah DEBOUNCE_DELAY menjadi 10 (10 ms), klik Build dan Upload, lalu tekan tombol lagi.

6. Ubah menjadi 200 (200 ms), klik Build dan Upload, lalu tekan tombol dengan cepat berulang kali.

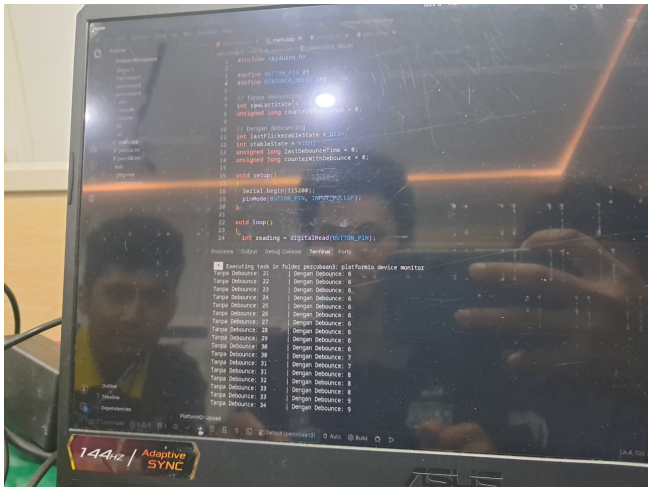
7. Bandingkan ketiga nilai (10, 50, dan 200 ms). Analisis untung ruginya: makin besar nilainya makin bersih dari getaran, tetapi tombol makin lambat merespons.



Gambar 8: Hasil Percobaan



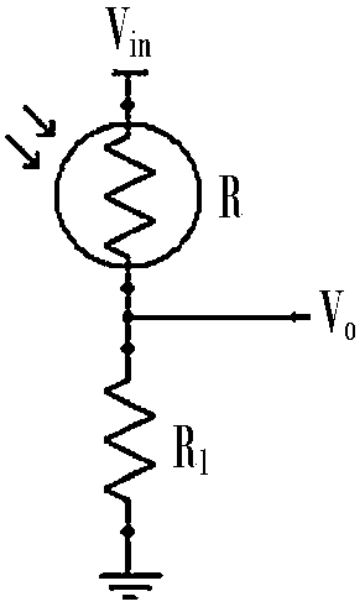
Gambar 9: Hasil Percobaan



Gambar 10: Hasil Percobaan

1.5 Percobaan 5 : Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

1. Pasang LDR bersama resistor sesuai skema, lalu hubungkan titik tengah keduanya ke GPIO 34.



Gambar 11: Skematik Rangkaian

2. Tulis program yang membaca nilai ADC mentah dan mengubahnya menjadi tegangan.

Listing 9: Program Pembacaan Nilai ADC pada Sensor LDR ESP32

```
#include <Arduino.h>

#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

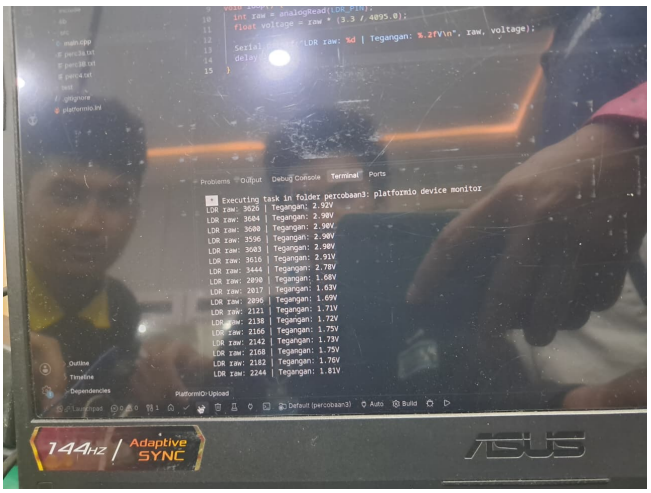
void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}
```

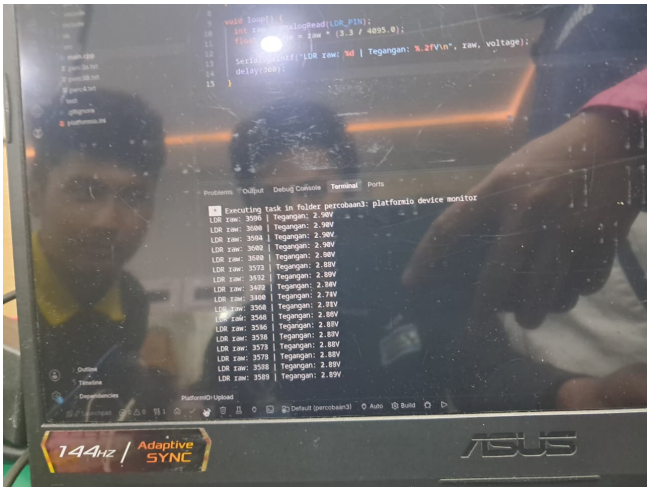
3. Klik Build dan Upload, lalu buka Serial Monitor.

4. Tutup LDR dengan tangan, lalu sinari dengan lampu. Lihat nilai mentahnya (0 sampai 4095) berubah.

5. Analisis mengapa sinyal analog bisa menunjukkan banyak kondisi, tidak hanya HIGH atau LOW.



Gambar 12: Hasil Percobaan



Gambar 13: Hasil Percobaan

2 Analisis Hasil Percobaan

Berdasarkan hasil dari kelima percobaan, seluruh percobaan berhasil dilakukan sesuai dengan tujuan. Pada percobaan pertama, STM32 Blackpill berhasil menjalankan program Blink menggunakan framework Arduino. Hal ini terjadi karena mikrokontroler menjalankan instruksi pada fungsi loop() secara berulang, yaitu memberikan logika LOW dan HIGH pada pin PC13 dengan jeda 500 ms. Karena LED onboard STM32 menggunakan konfigurasi active LOW, LED akan menyala ketika pin diberi logika LOW dan mati ketika diberi logika HIGH. Pada percobaan kedua, ESP32 berhasil menjalankan program Blink menggunakan framework ESP-IDF. Proses ini terjadi ketika fungsi app_main() mengatur GPIO sebagai output, lalu memberikan logika HIGH dan LOW bergantian. Fungsi vTaskDelay() digunakan untuk memberikan jeda selama 500 ms sehingga LED dapat berkedip dengan periodik. Pada percobaan ketiga, pengujian pull-up dan pull-down eksternal maupun internal berhasil menunjukkan perubahan logika ketika pushbutton ditekan dan dilepaskan. Pada konfigurasi pull-up, resistor menghubungkan pin GPIO ke tegangan 3,3 V sehingga kondisi awal pin bernilai HIGH. Ketika tombol ditekan, pin terhubung ke GND sehingga akan terbaca LOW. Sebaliknya, pada konfigurasi pull-down, resistor menghubungkan GPIO ke GND sehingga kondisi awal LOW, lalu berubah menjadi HIGH ketika tombol ditekan karena terhubung ke 3,3 V. Prinsip ini juga berlaku pada resistor internal ESP32,

tetapi jalur resistornya diaktifkan melalui konfigurasi INPUT_PULLUP dan INPUT_PULLDOWN pada program. Pada percobaan keempat, pengujian debouncing berhasil dilakukan dengan membandingkan pembacaan pushbutton tanpa debouncing dan dengan debouncing. Ketika tombol ditekan, kontak logam di dalamnya akan mengalami getaran kecil atau bouncing sehingga menghasilkan beberapa perubahan logika HIGH dan LOW dalam waktu singkat. Tanpa debouncing, setiap perubahan dari HIGH ke LOW dapat terdeteksi sebagai penekanan baru sehingga penghitung berpotensi bertambah lebih dari satu kali. Sementara itu, dengan debouncing, program memeriksa kestabilan sinyal selama lebih dari 50 ms sebelum menganggap perubahan tersebut sebagai penekanan yang valid. Oleh karena itu, pembacaan tombol menjadi lebih stabil dan mengurangi kemungkinan terjadinya penghitungan ganda. Pada percobaan kelima, pembacaan sensor LDR menggunakan ADC ESP32 berhasil dilakukan. Ketika intensitas cahaya berubah, resistansi LDR juga mengalami perubahan. Perubahan resistansi tersebut menyebabkan perubahan tegangan pada rangkaian pembagi tegangan yang terhubung ke pin GPIO 34. Tegangan analog yang masuk kemudian dikonversi oleh ADC ESP32 menjadi nilai digital dengan rentang 0–4095 karena menggunakan resolusi 12-bit. Nilai digital tersebut selanjutnya dikonversi menjadi perkiraan tegangan menggunakan perhitungan program dan ditampilkan melalui Serial Monitor.

3 Hasil Tugas Modul

3.1 Soal

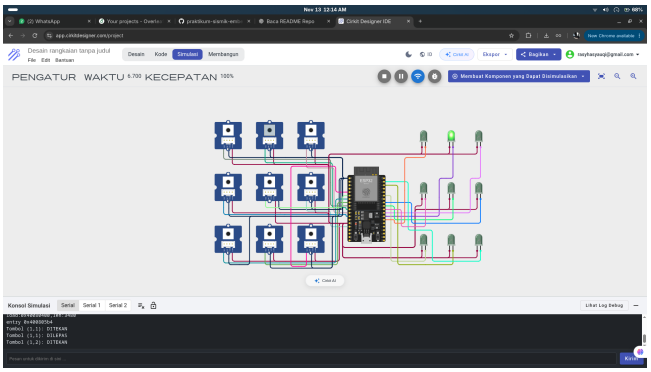
1. Buatlah simulasi untuk Matrix Pushbutton 3x3 yang mengendalikan Matrix LED 3x3, dimana tombol pada posisi (baris, kolom) tertentu menyalakan LED pada posisi yang sama, mis. tombol baris 1 kolom 1 menyalakan LED baris 1 kolom 1. Program wajib menerapkan debouncing pada setiap tombol (mengacu pada Percobaan 4), sehingga LED hanya berubah status setelah pembacaan tombol benar-benar stabil, bukan langsung mengikuti setiap perubahan sinyal mentah yang masih mungkin bouncing.

2. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing.

3. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?.

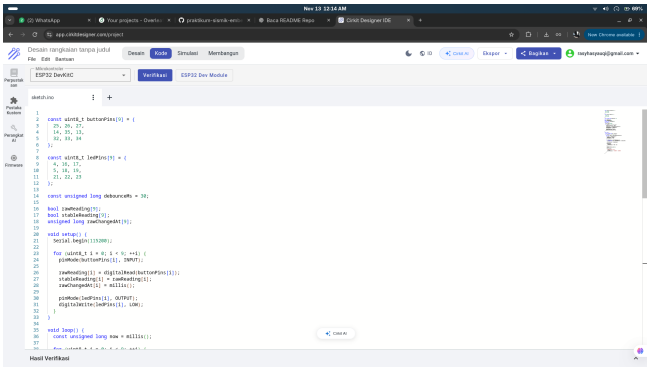
4. Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

3.2 Jawaban



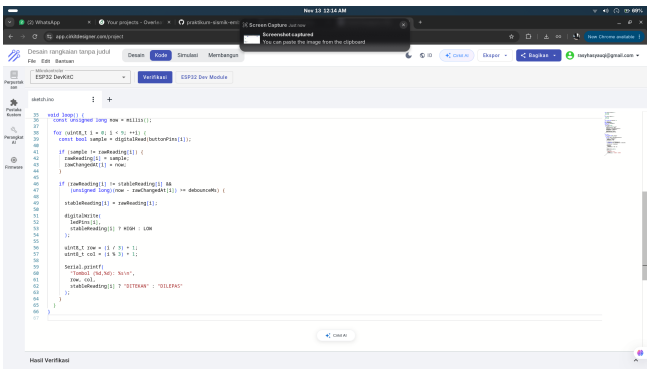
Gambar 14: Bukti Simulasi

1.



Gambar 15: Source Code

2.



Gambar 16: Source Code

3.

3.3 Pertanyaan Analisis

3.3.1 Pertanyaan

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing

pada C.5 Debouncing

2. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?

3. Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

3.4 Jawaban

1. Debouncing tetap diperlukan pada rangkaian matrix pushbutton walaupun biasanya hanya satu tombol yang ditekan dalam satu waktu karena setiap pushbutton menggunakan kontak mekanis yang bisa mengalami getaran atau bouncing ketika ditekan ataupun dilepaskan. Getaran tersebut menyebabkan kontak tombol terhubung dan terputus beberapa kali dalam waktu singkat yang menghasilkan perubahan sinyal HIGH dan LOW yang tidak stabil. Akibatnya, mikrokontroler dapat mendeteksi satu kali penekanan sebagai beberapa penekanan sekaligus dan menyebabkan LED berkedip atau berubah status secara tidak sesuai. Pada rangkaian matrix pushbutton, kondisi ini dapat memengaruhi ketepatan pembacaan koordinat tombol dan pengendalian LED yang bersesuaian. Oleh karena itu, debouncing digunakan untuk memastikan perubahan sinyal telah stabil selama waktu tertentu sebelum dianggap sebagai input yang valid, sehingga LED hanya berubah status sesuai penekanan tombol yang sebenarnya.

2. Pada rangkaian matrix pushbutton ini digunakan konfigurasi pull-up pada pembacaan kolom tombol. Ketika tombol tidak ditekan, pin kolom berada pada kondisi HIGH karena terhubung ke tegangan positif melalui resistor pull-up. Saat tombol ditekan, pin kolom terhubung ke GND melalui jalur baris yang sedang dipindai sehingga terbaca LOW. Kondisi pin tidak boleh dibiarkan floating karena tegangan pada pin menjadi tidak terdefinisi dan bisa menyebabkan pembacaan logika berubah ubah walau tombol tidak ditekan. Oleh karena itu, konfigurasi pull-up diperlukan untuk menjaga kondisi logika pin tetap stabil sehingga ESP32 dapat membedakan kondisi tombol ditekan dan tidak ditekan dengan benar.

3. Pada rangkaian ini, tombol menggunakan digital input, sedangkan LDR menggunakan analog input (ADC). Digital input hanya mengenali dua kondisi logika, yaitu HIGH dan LOW, sehingga hanya dapat membedakan kondisi tombol ditekan atau tidak ditekan. Sebaliknya, analog input dapat membaca variasi tegangan dari LDR yang berubah sesuai intensitas cahaya. Tegangan tersebut kemudian dikonversi oleh ADC menjadi nilai digital dengan beberapa tingkatan. Jika menggunakan ADC 12-bit seperti pada ESP32, terdapat $2^{12} = 4096$ tingkat pembacaan, yaitu dari 0 hingga 4095. Oleh karena itu, ADC mampu merepresentasikan lebih dari dua kondisi karena tidak hanya mengenali logika HIGH dan LOW, tetapi juga mengukur besarnya tegangan analog secara bertingkat.

4 Kesimpulan

Berdasarkan kelima percobaan pada modul 1, dapat disimpulkan bahwa STM32 Blackpill dan ESP32 bisa menjalankan program sesuai dengan konfigurasi dan framework yang digunakan. Percobaan pull-up dan pull-down memperlihatkan bagaimana konfigurasi resistor internal maupun eksternal memengaruhi pembacaan logika pada GPIO. Selain itu, penerapan debouncing juga terbukti penting untuk mengurangi kesalahan pembacaan akibat getaran mekanis pada push button. Pada percobaan ADC, ESP32 dapat membaca perubahan intensitas cahaya melalui sensor LDR dan mengubahnya menjadi nilai digital serta tegangan. Secara keseluruhan, percobaan ini memberikan pemahaman mengenai dasar pemrograman mikrokontroler, penggunaan GPIO, pengolahan input digital, dan pembacaan sinyal analog.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 17: Dokumentasi Saat Praktikum